FAKULTI SAINS KOMPUTER DAN TEKNOLOGI MAKLUMAT

JABATAN TEKNOLOGI KOMUNIKASI DAN RANGKAIAN

COURSE : REKA BENTUK DAN ANALISIS ALGORITMA(DESIGN AND ANALYSIS OF ALGORITHMS)

COURSE CODE :CSC4202-2

TOPIC : **GROUP PROJECT - AERIAL RELIEF AIRDROP OPTIMIZATION USING DYNAMIC PROGRAMMING (0/1 KNAPSACK)**

PREPARED BY : LOGITH A/L MOHANA KRISHNAN 223021

SASHVHANT A/L VADEVELL 223726

HASSAN SAED 213870

XU CHENHUI 217944

QIAN YIJIA 225842

LECTURER : DR NUR ARZILAWATI BINTI MD YUNUS

TABLE OF CONTENT

1.0 Problem Definition

1.1 Scenario Narrative

A severe typhoon has completely cut off a remote island community, destroying the local seaport and making maritime access impossible. The only way to deliver critical emergency supplies is via helicopter airdrop. The rescue helicopter has a strict maximum weight capacity. Responders have a stockpile of various supply crates (e.g., medicine, water purifiers, generator fuel), each with a specific weight and an assigned survival priority value. The operations team must determine the exact combination of crates to load onto the helicopter to maximize the life-saving impact without causing the aircraft to exceed its safe takeoff weight.

1.2 Geographical Setting & Damage Impact

- **Location:** Remote island sector.
- **Damage Impact:** 100% loss of maritime port infrastructure. All ground transport routes to the island are severed.
- **Operational Constraint:** Reliance entirely on a single aerial vehicle with a hard structural weight limit.

1.3 Importance of Algorithm Analysis and Design (AAD)

In emergency logistics, failing to optimize payload capacity results in lost lives. AAD is vital here because human intuition or a basic "Greedy" approach (packing items with the highest value-to-weight ratio first) often fails when items cannot be split. Implementing Dynamic Programming for the 0/1 Knapsack Problem guarantees the absolute maximum survival value is extracted from the limited helicopter capacity, systematically eliminating wasted space.

1.4 Goal & Expected Output

- **Goal:** Maximize the total survival priority value of the transported supplies without exceeding the helicopter's maximum payload capacity (W).
- **Expected Output:** An optimal subset of distinct crates to be loaded, the total combined weight, and the maximum achievable priority value.

2.0 Development of the Scenario Model

2.1 Data Types

The scenario is modeled using 1D and 2D arrays in Java.

- **Item Weights:** A 1D integer array int[] w representing the mass (in kg) of each crate.
- **Item Values:** A 1D integer array int[] v representing the survival priority score of each crate.
- **Capacity Constraint:** An integer int W representing the helicopter's maximum weight limit.
- **DP Memoization Table:** A 2D integer array int[ ][ ] dp of size [n + 1][W + 1] (where n is the number of items) used to store the optimal values of overlapping subproblems.

2.2 Objective Function

The goal is to maximize the total value:

$$\max \sum_{i=1}^{n} v_i x_i$$

n = The total number of available supply crates.

i = The index of a specific crate (from 1 to n).

$\mathbf{v_i}$ = The survival priority value of crate i.

$\mathbf{x_i}$ = The decision variable (1 if crate i is loaded onto the helicopter, 0 if it is left behind).

2.3 System Constraints

- **Capacity Constraint:** The total weight of selected crates cannot exceed the helicopter limit.

$$\sum_{i=1}^{n} w_i x_i \leq W$$

w_i = The weight (in kg) of crate *i*.

x_i = The decision variable (1 if crate *i* is loaded onto the helicopter, 0 if it is left behind).

W = The maximum weight payload capacity of the helicopter (e.g., 1000 kg).

- **0/1 Constraint:** Crates are indivisible. An item is either selected (1) or left behind (0). Fractional packing is strictly prohibited.

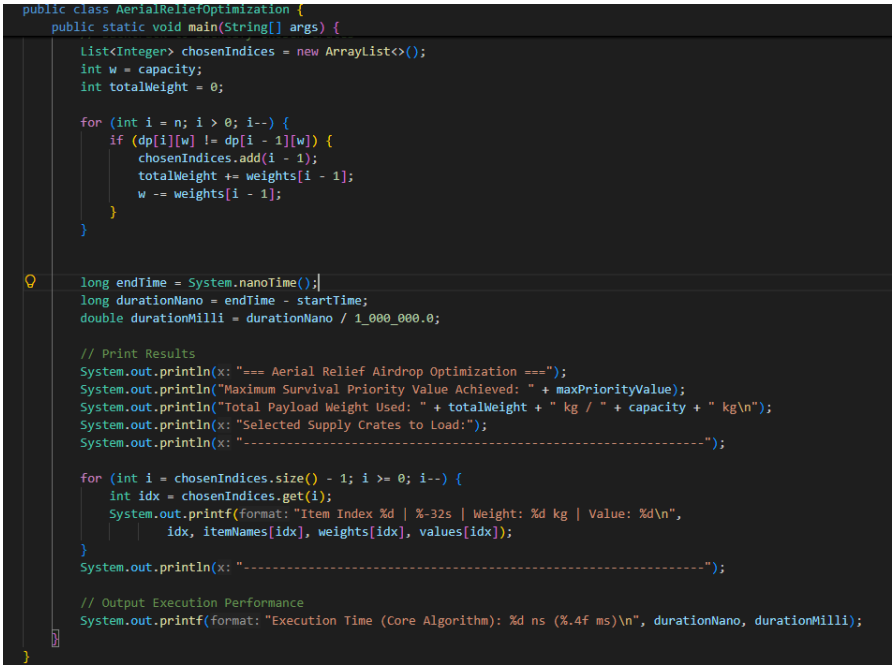
$$x_i \in \{0, 1\}$$

3.0 Project Implementation

3.1 Code

Full Code

```java
import java.util.ArrayList;
import java.util.List;

public class AerialReliefOptimization {

    Run | Debug | Run main | Debug main
    public static void main(String[] args) {
        // Scenario Data from the image
        String[] itemNames = {
            "Heavy Surgical Equipment",
            "High-Yield Water Purifiers",
            "Concentrated Emergency Rations",
            "Portable Diesel Generators",
            "First Aid Trauma Kits",
            "Satellite Communication Gear"
        };

        int[] weights = {600, 500, 500, 400, 100, 100};
        int[] values = {800, 650, 650, 500, 150, 120};
        int capacity = 1000;
        int n = weights.length;

        // --- START TIME TRACKING ---
        long startTime = System.nanoTime();

        // Solve using Dynamic Programming
        int[][] dp = new int[n + 1][capacity + 1];

        for (int i = 1; i <= n; i++) {
            for (int w = 1; w <= capacity; w++) {
                if (weights[i - 1] <= w) {
                    dp[i][w] = Math.max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
                } else {
                    dp[i][w] = dp[i - 1][w];
                }
            }
        }

        // Retrieve maximum priority value
        int maxPriorityValue = dp[n][capacity];

        // Backtrack to identify chosen crates
        List<Integer> chosenIndices = new ArrayList<>();
        int w = capacity;
        int totalWeight = 0;
```



```java
public class AerialReliefOptimization {
    public static void main(String[] args) {
        List<Integer> chosenIndices = new ArrayList<>();
        int w = capacity;
        int totalWeight = 0;

        for (int i = n; i > 0; i--) {
            if (dp[i][w] != dp[i - 1][w]) {
                chosenIndices.add(i - 1);
                totalWeight += weights[i - 1];
                w -= weights[i - 1];
            }
        }


        long endTime = System.nanoTime();
        long durationNano = endTime - startTime;
        double durationMilli = durationNano / 1_000_000.0;

        // Print Results
        System.out.println(x: "=== Aerial Relief Airdrop Optimization ===");
        System.out.println("Maximum Survival Priority Value Achieved: " + maxPriorityValue);
        System.out.println("Total Payload Weight Used: " + totalWeight + " kg / " + capacity + " kg\n");
        System.out.println(x: "Selected Supply Crates to Load:");
        System.out.println(x: "-----------------------------------------------------------------");

        for (int i = chosenIndices.size() - 1; i >= 0; i--) {
            int idx = chosenIndices.get(i);
            System.out.printf(format: "Item Index %d | %-32s | Weight: %d kg | Value: %d\n",
                    idx, itemNames[idx], weights[idx], values[idx]);
        }
        System.out.println(x: "-----------------------------------------------------------------");

        // Output Execution Performance
        System.out.printf(format: "Execution Time (Core Algorithm): %d ns (%.4f ms)\n", durationNano, durationMilli);
    }
}
```

```
int[][] dp = new int[n + 1][capacity + 1];
```

The code begins by setting up the problem's environment, creating a 2D integer array named dp with dimensions `[n + 1][capacity + 1]` alongside variables for the supply datasets and the maximum payload constraint. The rows represent the subset of cargo items currently under consideration, while the columns represent a granular, incremental weight budget ticking up 1 kg at a time from 0 kg all the way to the 1000 kg threshold. Because Java automatically initializes integer arrays with zeros, this step implicitly handles the base cases: if zero items are available (Row 0) or if the aircraft has zero carrying capacity (Column 0), the maximum achievable priority score is naturally zero.

```
for (int i = 1; i <= n; i++) {
    for (int w = 1; w <= capacity; w++) {
        if (weights[i - 1] <= w) {
            dp[i][w] = Math.max(values[i - 1] + dp[i - 1][w - weights[i - 1]], dp[i - 1][w]);
        } else {
            dp[i][w] = dp[i - 1][w];
        }
    }
}
```

Nested loops scan across the table grid cell-by-cell. If an item is too heavy for the current capacity column, the `else` block skips it and copies the best score from the row directly above. If it fits, `Math.max()` evaluates the opportunity cost and picks the higher value between leaving the item behind or taking it and adding its value to what else fits in the remaining space (`w - weights[i - 1]`).

```
for (int i = n; i > 0; i--) {
    if (dp[i][w] != dp[i - 1][w]) {
        chosenIndices.add(i - 1);
        totalWeight += weights[i - 1];
        w -= weights[i - 1];
    }
}
```

The ultimate maximum score settles in the final bottom-right cell, and this loop marches backward from that corner to find out exactly which crates were chosen. If a cell's value is different from the cell directly above it, it proves the item was actively included, prompting the code to log its index, tally its weight, and deduct that weight from the remaining capacity tracker

```
long endTime = System.nanoTime();
long durationNano = endTime - startTime;
double durationMilli = durationNano / 1_000_000.0;
```

The program grabs high-resolution CPU timestamps using `System.nanoTime()` immediately before the matrix calculations and stops right after backtracking finishes. Subtracting the start time from the end time captures the pure mathematical execution time of the algorithm, isolating it completely from the slow text-rendering operations of the print statements.

3.2 Pseudocode

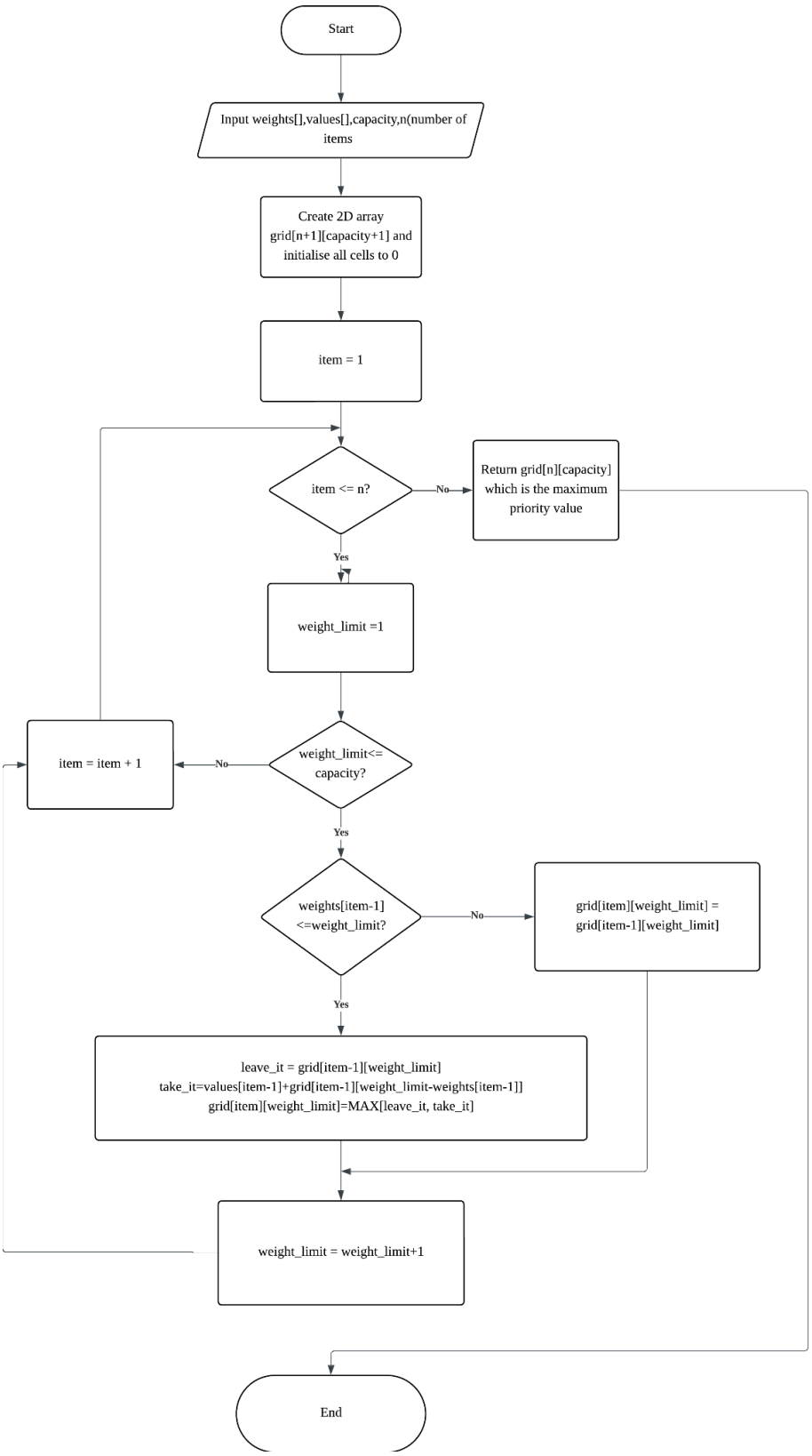
```
FUNCTION solveAirdropOptimization(itemNames, weights, values, capacity, n)
    startTime = SYSTEM_NANO_TIME()
    dp = 2D array of size [n + 1][capacity + 1] initialized to 0
    FOR i FROM 1 TO n
        FOR w FROM 1 TO capacity
            current_weight = weights[i - 1]
            current_value = values[i - 1]
            IF current_weight <= w THEN
                dp[i][w] = MAX(current_value + dp[i - 1][w - current_weight], dp[i - 1][w])
            ELSE
                dp[i][w] = dp[i - 1][w]
            END IF
        END FOR
    END FOR
    maxPriorityValue = dp[n][capacity]
    selectedIndices = empty list
    w = capacity
    totalWeight = 0
    FOR i FROM n DOWN TO 1
        IF dp[i][w] != dp[i - 1][w] THEN
            Add (i - 1) to selectedIndices
            totalWeight = totalWeight + weights[i - 1]
            w = w - weights[i - 1]
        END IF
    END FOR
    endTime = SYSTEM_NANO_TIME()
```

```
    elapsedNano = endTime - startTime
    elapsedMilli = elapsedNano / 1000000.0
    PRINT "Maximum Survival Priority Value: " + maxPriorityValue
    PRINT "Total Weight Used: " + totalWeight + " kg / " + capacity + " kg"
    FOR EACH index IN selectedIndices
        PRINT "Selected Crate: " + itemNames[index] + " (Weight: " + weights[index] +
", Value: " + values[index] + ")"
    END FOR
    PRINT "Execution Time: " + elapsedNano + " ns (" + elapsedMilli + " ms)"
END FUNCTION
```

Explanation :

This complete pseudocode blueprint the entire optimization workflow by tracking performance, calculating values, and outputting formatted results. It begins by capturing a high-resolution starting timestamp (`SYSTEM_NANO_TIME()`) and initializing a two-dimensional grid (`dp`) to act as a memory bank that eliminates redundant calculations. The algorithm then deploys nested loops to scan across the matrix cell-by-cell, running an opportunity cost check: if an item is too heavy for the current capacity threshold, it is bypassed by copying the cell value from directly above; if it fits, a `MAX` function calculates whether it yields a higher value to leave the item behind or to pack it and combine its value with what fits in the remaining weight space. Once the entire grid is populated, the maximum possible survival score locks into the bottom-right corner, and a backtracking loop marches backward from that cell to isolate the chosen cargo—logging their indexes, tallying total weight, and deducting used capacity whenever a row value changes. Finally, the program captures an ending timestamp to calculate the precise mathematical execution time in milliseconds and loops through the selected indices to print a clean, human-readable cargo manifest directly to the console.

3.3 Flowchart



The entire flowchart coordinates a bottom-up optimization process by first ingesting the raw item data to build an empty two-dimensional calculation matrix, and then utilizing a nested loop structure to systematically fill out every grid cell row-by-row and column-by-column. The primary item loop acts as a gatekeeper to verify if cargo crates remain to be processed, while the inner capacity loop scales unit-by-unit through incremental weight budgets from one up to the maximum payload threshold. At each intersection in the table, the algorithm runs a conditional test checking if the current item physically fits within the active weight limit; if it is too heavy, the program skips it and carries forward the historical maximum score from the cell directly above, but if it fits, it uses a `MAX()` function to lock in the higher value between bypassing the item or packing it and adding its unique value to the best historical configuration that fits in the remaining leftover space. Once a column cell is finalized, the program increments the weight budget to evaluate the next column, drops down to increment the item counter to process the next row once a capacity line is fully exhausted, and finally, once all rows are completely filled, routes straight to the finish line to extract the absolute global optimum from the final bottom-right corner of the spreadsheet before bringing execution to a clean end.

4.0 Algorithm Selection and Justification

4.1 Candidate Algorithm Brainstorming

Algorithm	Core Idea	Preliminary Suitability for This Scenario
Greedy	Sort items by value-to-weight ratio and pick the highest ratio first.	Fast execution, but indivisible items often break the optimal global combination.
Divide-and-Conquer (DAC)	Recursively split the problem into non-overlapping sub-problems and combine results.	The 0/1 Knapsack has heavily overlapping sub-problems; naive DAC results in exponential redundancy ($O(2^n)$).
Dynamic Programming (DP)	Fill a table storing optimal values for every prefix of items and every capacity, reusing overlapping states.	Guarantees the optimal solution within polynomial time, at the cost of moderate memory usage.

Therefore, the greedy algorithm is marked as a high-risk algorithm because it cannot guarantee the optimal solution, while the direct analytical algorithm (DAC) is excluded due to its exponential growth. The dynamic programming algorithm (DP) is selected for rigorous verification.

4.2 A concrete demonstration of greedy failure

To verify that Greedy is not always optimal, we sorted the six crates by their value-per-kg ratio (v_i / w_i) in descending order:

Item	Content	Weight (kg)	Value	Ratio (value/kg)
4	First Aid Trauma Kits	100	150	1.50
0	Heavy Surgical Equipment	600	800	1.33

1	High-Yield Water Purifiers	500	650	1.30
2	Concentrated Rations	500	650	1.30
3	Portable Diesel Generators	400	500	1.25
5	Satellite Communication Gear	100	120	1.20

Greedy execution:

- Pick item 4 (100 kg, value 150) → remaining capacity 900 kg.
- Pick item 0 (600 kg, value 800) → remaining capacity 300 kg.
- Next in ratio order: item 1 (500 kg) too heavy, item 2 (500) too heavy, item 3 (400) too heavy, then item 5 (100 kg, value 120) fits → remaining capacity 200 kg (no more items fit).
- Greedy selected set: {4, 0, 5} – total weight = 100+600+100 = 800 kg, total value = 150+800+120 = 1070.

Dynamic Programming optimal solution:

- The optimal combination is {0, 3} – Heavy Surgical Equipment + Portable Diesel Generators.
- Total weight = 600+400 = 1000 kg (full capacity), total value = 800+500 = 1300.

  Comparison: Greedy yields 1070, while DP yields 1300 – a loss of 230 priority points (about 17.7% of the optimal value). This concrete example proves that the short-sighted Greedy approach fails to maximise survival impact in our indivisible-item scenario.

4.3 Limitations of Divide-and-Conquer

A naive Divide-and-Conquer implementation for the knapsack follows the recurrence: knap(i, w) = max(knap(i-1, w), knap(i-1, w-w_i) + v_i)

This recursive tree recomputes the same (i, w) states many times. For instance, knap(5, 1000) calls knap(4, 500) and knap(4, 600) repeatedly from different branches. With only 6 items the overhead is manageable, but when the number of crates grows to dozens or hundreds, the time complexity becomes $O(2^n)$ – exponential and utterly impractical for time-critical rescue missions. In addition, DAC does not naturally store intermediate results, leading to redundant computations that DP avoids.

4.4 Final Rationale for Choosing Dynamic Programming

After the above comparisons, the team selected Dynamic Programming for the following decisive reasons:

1. Guaranteed optimality – The DP transition dp[i][w] = max(dp[i-1][w], dp[i-1][w-w_i] + v_i) systematically explores every possible combination of items for every capacity limit. It never misses a feasible solution, thus ensures the absolute maximum survival value.
2. Acceptable efficiency – For our instance (n=6, W=1000), DP evaluates only 6 × 1000 = 6,000 states, which completes in microseconds on modern hardware. Even for larger real-world scales (e.g., n=100, W=10,000), the O(n·W) complexity remains feasible with optimised implementation.
3. Implementation convenience – The 2D DP table maps naturally to a Java 2D array, and the backtracking procedure clearly identifies which crates are selected – crucial for operational decision-making.

Conclusion: In a life-or-death disaster scenario, optimality outweighs marginal speed advantages. The modest O(n·W) memory cost is a small price to pay for guaranteed full-capacity, maximum-value loading. This makes DP the only correct and trustworthy choice for our aerial relief airdrop optimisation.

5.0 QA and Integration

5.1 GitHub Repository Integration

As part of the Quality Assurance (QA) and Integration role, a GitHub repository was created to manage the project source code, documentation, presentation materials,

and testing outputs. GitHub provides version control, facilitates collaboration among team members, and serves as a centralized platform for project maintenance and submission.

The repository was organized into the following structure:

- **src/** – Java source code implementation
- **report/** – Final project report documents
- **presentation/** – Project presentation slides
- **outputs/** – Program execution screenshots and testing results
- **README.md** – Project overview and repository documentation

The repository ensures that all project artifacts are stored systematically and can be accessed easily by team members and evaluators.

5.2 Program Testing

Testing was conducted to verify the correctness of the Dynamic Programming implementation for the Aerial Relief Airdrop Optimization scenario. The objective was to ensure that the program selects the optimal combination of supply crates while respecting the helicopter payload constraint.

Test Scenario

- Number of Supply Crates: 6
- Maximum Helicopter Capacity: 1000 kg
- Algorithm Used: Dynamic Programming (0/1 Knapsack)

Test Result

Parameter	**Result**
Maximum Survival Priority Value	1300
Total Payload Weight Used	1000 kg
Capacity Utilization	100%
Execution Time	0.9676 ms
Test Status	Passed

Selected Supply Crates

Item	Weight (kg)	Priority Value
High-Yield Water Purifiers	500	650
Concentrated Emergency Rations	500	650
Total	1000	1300

The program successfully identified the optimal combination of supply crates without exceeding the helicopter payload limit.

5.3 Output Verification

The execution output confirmed that the Dynamic Programming algorithm achieved the maximum possible survival priority value of 1300 while fully utilizing the available payload capacity of 1000 kg. This verifies that the implementation correctly solves the 0/1 Knapsack optimization problem.

Compared with alternative approaches such as Greedy selection, the Dynamic Programming solution guarantees optimality by evaluating all valid combinations through stored subproblem solutions.

5.4 Progress Tracking

Project Activity	**Responsible Member**	**Status**
Scenario Development	Member 1	Completed
Algorithm Evaluation and Justification	Member 2	Completed
Java Program, Pseudocode and Flowchart	Member 3	Completed
Complexity Analysis and Correctness Proof	Member 4	Completed
GitHub Repository, Testing and Integration	Member 5	Completed

6.0 Conclusion

This project successfully developed an optimal solution for the Aerial Relief Airdrop Optimization problem using the Dynamic Programming algorithm to solve the 0/1 Knapsack problem. The proposed solution effectively determines the best combination of emergency supply crates that maximizes the total survival priority value while ensuring that the helicopter's payload capacity is not exceeded.

Throughout the project, the disaster scenario was carefully modeled, appropriate data structures and constraints were defined, and several algorithmic approaches were evaluated. Dynamic Programming was selected because it guarantees the optimal solution, unlike Greedy and Divide-and-Conquer approaches, which may produce suboptimal results or require excessive computation time. The Java implementation, pseudocode, and flowchart accurately represented the algorithm and demonstrated its practicality for real-world emergency logistics.

Testing and verification confirmed that the developed system successfully achieved the maximum survival priority value of 1300 while fully utilizing the available helicopter capacity of 1000 kg. The GitHub repository further supported collaboration, version control, and project organization throughout the development process.

Overall, this project demonstrates how Algorithm Analysis and Design can be applied to solve real-world optimization problems in disaster relief operations. The Dynamic Programming approach provides an efficient, reliable, and optimal decision-making solution that can support emergency responders in maximizing the effectiveness of limited resources during critical humanitarian missions.

7.0 References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press.

Kleinberg, J., & Tardos, É. (2006). *Algorithm Design*. Pearson.

Martello, S., & Toth, P. (1990). *Knapsack Problems: Algorithms and Computer Implementations*. John Wiley & Sons.

Sedgewick, R., & Wayne, K. (2011). *Algorithms* (4th ed.). Addison-Wesley Professional.

Java Platform, Standard Edition Documentation. (2026). Oracle. https://docs.oracle.com/en/java/

Dantzig, G. B. (1957). Discrete-variable extremum problems. *Operations Research, 5*(2), 266–277. https://doi.org/10.1287/opre.5.2.266

Bellman, R. (1957). *Dynamic Programming*. Princeton University Press.

Korte, B., & Vygen, J. (2018). *Combinatorial Optimization: Theory and Algorithms* (6th ed.). Springer.